

Python for Finance - M1 Economie Finance - Partie 1

Université Paris Dauphine

L. Briens

Dauphine - CREST - Institut Polytechnique

Octobre 2024

1 Introduction

2 Premiers pas en Python

3 Les types de variables

- Variables constantes
- Built-in variables

Outline

- 1 Introduction
- 2 Premiers pas en Python
- 3 Les types de variables
 - Variables constantes
 - Built-in variables

Le langage Python

- Un langage de **haut niveau**.

Le langage Python

- Un langage de **haut niveau**.
- Un langage **orienté objet** (POO).

Le langage Python

- Un langage de **haut niveau**.
- Un langage **orienté objet** (POO).
- Un langage **interprété** (et non compilé).

Le langage Python

- Un langage de **haut niveau**.
- Un langage **orienté objet** (POO).
- Un langage **interprété** (et non compilé).
- Un langage **clair** et **facile** à programmer.

Le langage Python

- Un langage de **haut niveau**.
- Un langage **orienté objet** (POO).
- Un langage **interprété** (et non compilé).
- Un langage **clair** et **facile** à programmer.
- Un langage **décentralisé** (open-source et communautaire).

Le langage Python

- Un langage de **haut niveau**.
- Un langage **orienté objet** (POO).
- Un langage **interprété** (et non compilé).
- Un langage **clair** et **facile** à programmer.
- Un langage **décentralisé** (open-source et communautaire).
- 3 paradigmes :
 - **Objet**
 - **Impératif**
 - **Fonctionnel**

Le langage Python

- Un langage de **haut niveau**.
- Un langage **orienté objet** (POO).
- Un langage **interprété** (et non compilé).
- Un langage **clair** et **facile** à programmer.
- Un langage **décentralisé** (open-source et communautaire).
- 3 paradigmes :
 - **Objet**
 - **Impératif**
 - **Fonctionnel**

NB : Python est un langage indenté (contrairement à VBA), il est obligatoire de respecter l'indentation pour pouvoir exécuter un code.

Utilisation de Python

- Directement depuis le CMD.exe (après avoir installé Python) :

Utilisation de Python

- Directement depuis le CMD.exe (après avoir installé Python) :
- Avec un IDE (Integrated Development Environment)
 - PyCharm
 - Spyder
 - VScode
 - ...

Utilisation de Python

- Directement depuis le CMD.exe (après avoir installé Python) :
- Avec un IDE (Integrated Development Environment)
 - PyCharm
 - Spyder
 - VScode
 - ...
- Avec Anaconda. Anaconda est une distribution de python et R et non un IDE. Il propose une structure de gestion des packages et des environnements.

Utilisation de Python

- Directement depuis le CMD.exe (après avoir installé Python) :
- Avec un IDE (Integrated Development Environment)
 - PyCharm
 - Spyder
 - VScode
 - ...
- Avec Anaconda. Anaconda est une distribution de python et R et non un IDE. Il propose une structure de gestion des packages et des environnements.
- Avec Jupyter Notebook (ou JupyterLab). Des applications web permettant de développer en Python, Julia, R, Ruby, Scala... Permet de coder dans des Notebooks : Ceux-ci combinent des passages de code et des passages de Markdown.
NB : Jupyter utilise le noyau IPython qui permet d'autres syntaxes, de l'auto-complétion de code, des "magics commands" ...

Tout est objet (1/2)

En Python tout est **objet**, les objets, les fonctions, les modules, les méthodes. La dimension objet de python est sa structure de base c'est ainsi que le langage doit être abordé.

Dès qu'une variable est créée, elle référence un objet, que ce soit un nombre entier (int), un nombre flottant (float), un tableau (list) ou toute autre structure.

Tout est objet (1/2)

En Python tout est **objet**, les objets, les fonctions, les modules, les méthodes. La dimension objet de python est sa structure de base c'est ainsi que le langage doit être abordé.

Dès qu'une variable est créée, elle référence un objet, que ce soit un nombre entier (int), un nombre flottant (float), un tableau (list) ou toute autre structure.

Définition

En informatique, un objet est une modélisation d'un élément du monde réel. Il s'agit en fait de données et de fonctions qui agissent sur ces données réunies dans une même entité. Ces données sont appelées les **arguments** de l'objet. Les fonctions sont les **méthodes**. On peut se représenter chaque objet comme une capsule qui regroupe des attributs et des méthodes.

objet = attributs + méthodes

Tout est objet (2/2)

Chaque objet appartient au moins à une **classe**. Une classe permet de créer des objets. C'est une famille d'objets équivalents qui représentent un type de données. Un objet, aussi appelé une **instance**, est un exemplaire particulier d'une classe.

Tout est objet (2/2)

Chaque objet appartient au moins à une **classe**. Une classe permet de créer des objets. C'est une famille d'objets équivalents qui représentent un type de données. Un objet, aussi appelé une **instance**, est un exemplaire particulier d'une classe.

Exemple

Une classe peut être la classe "Voiture", quelque-uns de ses **attributs** seront :

- Roues
- Cylindrée
- Couleur

Quelque-unes de ses **méthodes** seront :

- Rouler
- Démarrer
- Mettre le clignotant

Une **instance** de cette classe sera : une unique voiture avec ses caractéristiques.

Taxonomie des objets en python

- **IDE** : (*Integrated Development Environment*) est une interface qui permet de développer, compiler et exécuter un programme dans un langage donné.

Taxonomie des objets en python

- **IDE** : (*Integrated Development Environment*) est une interface qui permet de développer, compiler et exécuter un programme dans un langage donné.
- **Module** : Ensemble de données et de fonctions réutilisables enregistrés dans un fichier texte d'extension ".py". Il peut être codé par l'utilisateur ou importé (*exemple : datetime, math, os ...*).

Taxonomie des objets en python

- **IDE** : (*Integrated Development Environment*) est une interface qui permet de développer, compiler et exécuter un programme dans un langage donné.
- **Module** : Ensemble de données et de fonctions réutilisables enregistrés dans un fichier texte d'extension ".py". Il peut être codé par l'utilisateur ou importé (*exemple : datetime, math, os ...*).
- **Package** : Ensemble de modules regroupés en un "package" importable de la même façon qu'un module (*exemple : Pandas, numpy, matplotlib ...*).

Taxonomie des objets en python

- **IDE** : (*Integrated Development Environment*) est une interface qui permet de développer, compiler et exécuter un programme dans un langage donné.
- **Module** : Ensemble de données et de fonctions réutilisables enregistrés dans un fichier texte d'extension ".py". Il peut être codé par l'utilisateur ou importé (*exemple : datetime, math, os ...*).
- **Package** : Ensemble de modules regroupés en un "package" importable de la même façon qu'un module (*exemple : Pandas, numpy, matplotlib ...*).
- **Fonction** : Sous programme regroupant plusieurs instructions.

Taxonomie des objets en python

- **IDE** : (*Integrated Development Environment*) est une interface qui permet de développer, compiler et exécuter un programme dans un langage donné.
- **Module** : Ensemble de données et de fonctions réutilisables enregistrés dans un fichier texte d'extension ".py". Il peut être codé par l'utilisateur ou importé (*exemple : datetime, math, os ...*).
- **Package** : Ensemble de modules regroupés en un "package" importable de la même façon qu'un module (*exemple : Pandas, numpy, matplotlib ...*).
- **Fonction** : Sous programme regroupant plusieurs instructions.
- **Méthode** : Fonction rattachée/appartenant à une classe.

Les mots-clés (1/2)

Définition

Les **mots-clés** sont des mots prédéfinis et réservés. Ils ont une signification particulière et immuable pour l'interpréteur.

Les mots-clés (1/2)

Définition

Les **mots-clés** sont des mots prédéfinis et réservés. Ils ont une signification particulière et immuable pour l'interpréteur.

Il existe 2 types de mots-clés :

- **Les mots clés réservés** : vous n'avez pas le droit d'utiliser un tel mot clé pour définir un identifiant (nom de variable, de fonction, de classes, d'attribut de méthode...). Sa seule utilisation autorisée est l'instruction à laquelle il est rattaché.

Les mots-clés (1/2)

Définition

Les **mots-clés** sont des mots prédéfinis et réservés. Ils ont une signification particulière et immuable pour l'interpréteur.

Il existe 2 types de mots-clés :

- **Les mots clés réservés** : vous n'avez pas le droit d'utiliser un tel mot clé pour définir un identifiant (nom de variable, de fonction, de classes, d'attribut de méthode...). Sa seule utilisation autorisée est l'instruction à laquelle il est rattaché.
- **Les mots clés contextuels** : un tel mot clé prend un sens dans un contexte bien particulier. En dehors de ce contexte, le mot reste utilisable pour nommer une variable, une fonction... Cette possibilité est assez récente et s'explique par le fait qu'il est nécessaire de faire évoluer le langage (et notamment d'y ajouter de nouvelles instructions) sans produire de rupture de compatibilité avec les programmes écrits avant l'apparition de la nouvelle fonctionnalité.

Les mots-clés (2/2)

Pour obtenir la liste des mots-clés **réservés** :

```
help("keywords")
```

#help(any_key_word) - permet d'obtenir de l'aide sur un mot-clé

```
Entrée [4]: help("keywords")
```

Here is a list of the Python keywords. Enter any keyword to get more help.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Il y a 35 mots-clés réservés en python 3.10 et uniquement 2 mots clés contextuels apparus avec Python 3.10 (**match** et **case**).

Les mots-clés basiques

Exemples :

**Création d'un module et utilisation de mots clés
basiques**

Outline

- 1 Introduction
- 2 Premiers pas en Python
- 3 Les types de variables
 - Variables constantes
 - Built-in variables

Téléchargement

Télécharger **Anaconda** :

- [Cliquer ici](#)

Télécharger **Pycharm** (version gratuite) :

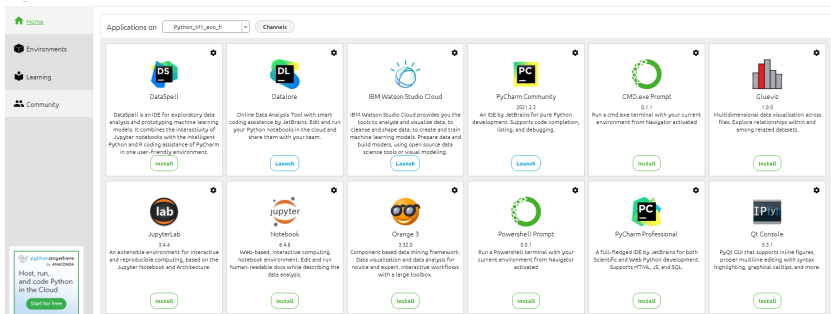
- [Cliquer ici](#)

Optionnel : Télécharger un **noyau python** (Version 3.8 et +) :

- [Cliquer ici](#)

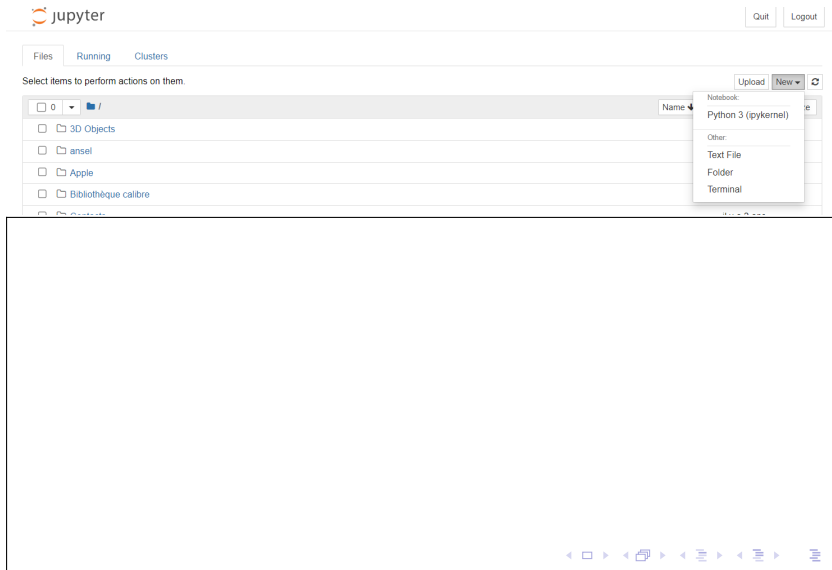
Anaconda

Ouvrir d'Anaconda Navigator et installer Jupyter Notebook, Spyder et CMD.exe Prompt :



Jupyter

Créer un premier Notebook :



The screenshot displays the Jupyter web interface. At the top, the Jupyter logo is on the left, and 'Quit' and 'Logout' buttons are on the right. Below the logo, there are tabs for 'Files', 'Running', and 'Clusters'. A message states 'Select items to perform actions on them.' To the right of this message are 'Upload', 'New', and a refresh icon. The 'New' dropdown menu is open, showing options: 'Notebook: Python 3 (ipykernel)', 'Other: Text File', 'Folder', and 'Terminal'. The file browser on the left shows a directory structure with folders like '3D Objects', 'ansel', 'Apple', 'Bibliothèque calibre', and 'Documents'.

Jupyter

Quit Logout

Files Running Clusters

Select items to perform actions on them.

Upload New

Notebook:
Python 3 (ipykernel)

Other:
Text File
Folder
Terminal

0 /

3D Objects

ansel

Apple

Bibliothèque calibre

Documents

Navigation icons: back, forward, search, etc.

Outline

- 1 Introduction
- 2 Premiers pas en Python
- 3 Les types de variables
 - Variables constantes
 - Built-in variables

Vue d'ensemble des différents types de variables

Les variables constantes

En python nous retrouvons les types basiques des autres langages tels que les entiers (**int**), les décimaux (**float**), les chaînes de caractères (**str**) et les booléens (**bool**).

Vue d'ensemble des différents types de variables

Les variables constantes

En python nous retrouvons les types basiques des autres langages tels que les entiers (**int**), les décimaux (**float**), les chaînes de caractères (**str**) et les booléens (**bool**).

Les "Built-in" variables

En python nous retrouvons aussi les types plus complexes tels que les listes (**list**), les tuples (**tuple**), les dictionnaires (**dict**) et les ensembles (**set**).

Outline

- 1 Introduction
- 2 Premiers pas en Python
- 3 Les types de variables
 - Variables constantes
 - Built-in variables

String

Le type String est noté **str** en Python.

Il permet de représenter des lettres et des mots, il n'existe pas en python de différenciation des lettres et des chaînes de caractères comme dans certains autres langages.

```
my_str = "Dauphine"  
print(my_str)  
==> Dauphine
```

```
my_str2 = 'dauphine'  
print(my_str2)  
==> dauphine
```

```
my_str == my_str2  
==> False
```

String

Le type String est noté **str** en Python.

Il permet de représenter des lettres et des mots, il n'existe pas en python de différenciation des lettres et des chaînes de caractères comme dans certains autres langages.

```
my_str = "Dauphine"  
print(my_str)  
==> Dauphine
```

```
my_str2 = 'dauphine'  
print(my_str2)  
==> dauphine
```

```
my_str == my_str2  
==> False
```

Caractéristique

- Les strings se déclarent avec des simples ou doubles quotes (' ou ").
- Elles sont sensibles à la casse.
- Les espaces sont des caractères.

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"
print(len(my_str))
==> ?
print(my_str.find("i"))
==> ?
print(my_str.capitalize(),my_str.upper(),my_str.lower())
==> ?
print(my_str.count("i"))
==> ?
print(my_str.replace("Dauphine", "9"))
==> ?
print(my_str.isdigit(),my_str.isalpha())
==> ?
print(my_str.split())
==> ?
print(my_str[slice(11,-9)])
==> ?
```

Question

Quels seront les outputs de ces méthodes de la classe str ?

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"  
print(len(my_str))
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"  
print(len(my_str))  
==> 25  
print(my_str.find("i"))
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"  
print(len(my_str))  
==> 25  
print(my_str.find("i"))  
==> 2  
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"
```

```
print(len(my_str))
```

```
==> 25
```

```
print(my_str.find("i"))
```

```
==> 2
```

```
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

```
Université Paris Dauphine UNIVERSITÉ PARIS DAUPHINE université paris dauphine
```

```
print(my_str.count("i"))
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"
```

```
print(len(my_str))
```

```
==> 25
```

```
print(my_str.find("i"))
```

```
==> 2
```

```
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

```
Université Paris Dauphine UNIVERSITÉ PARIS DAUPHINE université paris dauphine
```

```
print(my_str.count("i"))
```

```
==> 4
```

```
print(my_str.replace("Dauphine", "9"))
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"
```

```
print(len(my_str))
```

```
==> 25
```

```
print(my_str.find("i"))
```

```
==> 2
```

```
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

```
Université Paris Dauphine UNIVERSITÉ PARIS DAUPHINE université paris dauphine
```

```
print(my_str.count("i"))
```

```
==> 4
```

```
print(my_str.replace("Dauphine", "9"))
```

```
==> Université Paris 9
```

```
print(my_str.isdigit(),my_str.isalpha())
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"  
print(len(my_str))
```

```
==> 25
```

```
print(my_str.find("i"))
```

```
==> 2
```

```
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

```
Université Paris Dauphine UNIVERSITÉ PARIS DAUPHINE université paris dauphine
```

```
print(my_str.count("i"))
```

```
==> 4
```

```
print(my_str.replace("Dauphine", "9"))
```

```
==> Université Paris 9
```

```
print(my_str.isdigit(),my_str.isalpha())
```

```
==> False False
```

```
print(my_str.split())
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"  
print(len(my_str))
```

```
==> 25
```

```
print(my_str.find("i"))
```

```
==> 2
```

```
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

```
Université Paris Dauphine UNIVERSITÉ PARIS DAUPHINE université paris dauphine
```

```
print(my_str.count("i"))
```

```
==> 4
```

```
print(my_str.replace("Dauphine", "9"))
```

```
==> Université Paris 9
```

```
print(my_str.isdigit(),my_str.isalpha())
```

```
==> False False
```

```
print(my_str.split())
```

```
==> ['Université', 'Paris', 'Dauphine']
```

```
print(my_str[slice(11,-9)])
```

Méthode de la classe string et fonctions utiles

```
my_str = "Université Paris Dauphine"
```

```
print(len(my_str))
```

```
==> 25
```

```
print(my_str.find("i"))
```

```
==> 2
```

```
print(my_str.capitalize(),my_str.upper(),my_str.lower())
```

```
Université Paris Dauphine UNIVERSITÉ PARIS DAUPHINE université paris dauphine
```

```
print(my_str.count("i"))
```

```
==> 4
```

```
print(my_str.replace("Dauphine", "9"))
```

```
==> Université Paris 9
```

```
print(my_str.isdigit(),my_str.isalpha())
```

```
==> False False
```

```
print(my_str.split())
```

```
==> ['Université', 'Paris', 'Dauphine']
```

```
print(my_str[slice(11,-9)])
```

```
==> Paris
```

Nombres entiers et décimaux

Les nombres peuvent être de type entiers ("Integer" noté **int**) ou décimaux ("Float" noté **float**).

```
my_int = 7
my_int2 = int(7.7)
my_float = 7.7
my_float2 = float(7)
print(my_int,my_int2,my_float,my_float2)
==> 7 7 7.7 7.0

type(my_int + my_float)
==> <class 'float'>

print("7.3+7.7 =",7.3+7.7, "de type :",type(7.7+7.3))
==> 7.3+7.7 = 15.0 de type : <class 'float'>
```

Nombres entiers et décimaux

Les nombres peuvent être de type entiers ("Integer" noté **int**) ou décimaux ("Float" noté **float**).

```
my_int = 7
my_int2 = int(7.7)
my_float = 7.7
my_float2 = float(7)
print(my_int,my_int2,my_float,my_float2)
==> 7 7 7.7 7.0

type(my_int + my_float)
==> <class 'float'>

print("7.3+7.7 =",7.3+7.7, "de type :",type(7.7+7.3))
==> 7.3+7.7 = 15.0 de type : <class 'float'>
```

NB : L'application de la fonction `int()` sur un float supprime toute partie décimale de ce dernier. Ce n'est pas un arrondi, une partie entière ou autre méthode de discrétisation d'une variable continue en variable discrète.

Manipulation de nombres

La manipulation des nombres en python est possible avec toutes les opérations usuelles. Il existe aussi des *built-in function* et des fonctions du module "math" (module natif) très utiles.

Manipulation de nombres

La manipulation des nombres en python est possible avec toutes les opérations usuelles. Il existe aussi des *built-in function* et des fonctions du module "math" (module natif) très utiles.

Built-in functions :

```
pow(base = 7, exp = 3)
round(7.8)
min(1,5,2.8)
max(1,5,2.8)
abs(-8)
```

Manipulation de nombres

La manipulation des nombres en python est possible avec toutes les opérations usuelles. Il existe aussi des *built-in function* et des fonctions du module "math" (module natif) très utiles.

Built-in functions :

```
pow(base = 7, exp = 3)
round(7.8)
min(1,5,2.8)
max(1,5,2.8)
abs(-8)
```

Quelques fonctions du module math :

```
import math
print(math.ceil(7.7),math.ceil(-7.7))
==> 8 -7
print(math.floor(7.7),math.floor(-7.7))
==> 7 -8
math.sqrt(64)
==> 8
```

Manipulation de nombres

La manipulation des nombres en python est possible avec toutes les opérations usuelles. Il existe aussi des *built-in function* et des fonctions du module "math" (module natif) très utiles.

Built-in functions :

```
pow(base = 7, exp = 3)
round(7.8)
min(1,5,2.8)
max(1,5,2.8)
abs(-8)
```

Quelques fonctions du module math :

```
import math
print(math.ceil(7.7),math.ceil(-7.7))
==> 8 -7
print(math.floor(7.7),math.floor(-7.7))
==> 7 -8
math.sqrt(64)
==> 8
```

NB : Le module math possède une grande quantité de fonctions, vous pouvez les retrouver [ici](#) .

Outline

- 1 Introduction
- 2 Premiers pas en Python
- 3 Les types de variables
 - Variables constantes
 - Built-in variables

Listes

Une liste est un objet de type container, les containers contiennent d'autres objets de tout type et ils sont ordonnés.

Déclaration d'une liste et affichage de cette dernière :

```
my_list = list()  
my_list = []  
print(my_list)
```

Listes

Une liste est un objet de type container, les containers contiennent d'autres objets de tout type et ils sont ordonnés.

Déclaration d'une liste et affichage de cette dernière :

```
my_list = list()  
my_list = []  
print(my_list)
```

Caractéristiques

Une liste est :

- **Ordonnée/indexée**
- **Mutable** (donc copie par référence, non-hashable)
- **Accessible**
- **Imbriquable**
- **Extensible**
- **Non-Hashable** (car mutable)

Manipuler des listes

Les listes étant ordonnées on les manipulent essentiellement par leurs indices mais on peut aussi les fusionner.

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']  
print(my_stocks[1], my_stocks[1:3], my_stocks[-2:], my_stocks[::2])
```

Manipuler des listes

Les listes étant ordonnées on les manipulent essentiellement par leurs indices mais on peut aussi les fusionner.

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(my_stocks[1], my_stocks[1:3], my_stocks[-2:], my_stocks[::2])
==> TSLA ['TSLA', 'AAPL'] ['AAPL', 'META'] ['AMZN', 'AAPL']

my_stocks2 = ['KO', 'P911']
my_stocks2 = my_stocks2 + my_stocks
print(my_stocks2)
```

Manipuler des listes

Les listes étant ordonnées on les manipulent essentiellement par leurs indices mais on peut aussi les fusionner.

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(my_stocks[1], my_stocks[1:3], my_stocks[-2:], my_stocks[::-2])
==> TSLA ['TSLA', 'AAPL'] ['AAPL', 'META'] ['AMZN', 'AAPL']
```

```
my_stocks2 = ['KO', 'P911']
my_stocks2 = my_stocks2 + my_stocks
print(my_stocks2)
==> ['KO', 'P911', 'AMZN', 'TSLA', 'AAPL', 'META']
```

```
copy_stocks = my_stocks
copy_stocks.pop()
print(copy_stocks, my_stocks)
```

Manipuler des listes

Les listes étant ordonnées on les manipulent essentiellement par leurs indices mais on peut aussi les fusionner.

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(my_stocks[1], my_stocks[1:3], my_stocks[-2:], my_stocks[::-2])
==> TSLA ['TSLA', 'AAPL'] ['AAPL', 'META'] ['AMZN', 'AAPL']

my_stocks2 = ['KO', 'P911']
my_stocks2 = my_stocks2 + my_stocks
print(my_stocks2)
==> ['KO', 'P911', 'AMZN', 'TSLA', 'AAPL', 'META']

copy_stocks = my_stocks
copy_stocks.pop()
print(copy_stocks, my_stocks)
['AMZN', 'TSLA', 'AAPL'] ['AMZN', 'TSLA', 'AAPL']
```

On remarque le caractère **non-hashable** des listes (car elles sont **mutables**). Ici la copie de la liste a été faite en **référence** et non en **valeur**.

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> ?
print(my_stocks.append('MSFT'), my_stocks)
==> ? ?
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> ? ?
print(my_stocks.count('MSFT'))
==> ?
print(my_stocks.pop(), my_stocks)
==> ? ?
print(my_stocks.remove('TSLA'), my_stocks)
==> ? ?
print(my_stocks.reverse(), my_stocks)
==> ? ?
print(my_stocks.extend(['MC', 'TTE']), my_stocks)
==> ? ?
print(my_stocks.sort(reverse=True), my_stocks)
==> ? ?
Print(my_stocks.index('MC'))
==> ?
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']  
print(len(my_stocks))
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']  
print(len(my_stocks))  
==> 4  
print(my_stocks.append('MSFT'), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']  
print(len(my_stocks))  
==> 4  
print(my_stocks.append('MSFT'), my_stocks)  
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']  
print(my_stocks.insert(1, 'FTKE'), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
==> 'MSFT' ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META']
print(my_stocks.remove('TSLA'), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
==> 'MSFT' ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META']
print(my_stocks.remove('TSLA'), my_stocks)
==> None ['AMZN', 'FTKE', 'AAPL', 'META']
print(my_stocks.reverse(), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
==> 'MSFT' ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META']
print(my_stocks.remove('TSLA'), my_stocks)
==> None ['AMZN', 'FTKE', 'AAPL', 'META']
print(my_stocks.reverse(), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN']
print(my_stocks.extend(['MC', 'TTE']), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
==> 'MSFT' ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META']
print(my_stocks.remove('TSLA'), my_stocks)
==> None ['AMZN', 'FTKE', 'AAPL', 'META']
print(my_stocks.reverse(), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN']
print(my_stocks.extend(['MC', 'TTE']), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN', 'MC', 'TTE']
print(my_stocks.sort(reverse=True), my_stocks)
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
==> 'MSFT' ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META']
print(my_stocks.remove('TSLA'), my_stocks)
==> None ['AMZN', 'FTKE', 'AAPL', 'META']
print(my_stocks.reverse(), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN']
print(my_stocks.extend(['MC', 'TTE']), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN', 'MC', 'TTE']
print(my_stocks.sort(reverse=True), my_stocks)
==> None ['TTE', 'META', 'MC', 'FTKE', 'AMZN', 'AAPL']
Print(my_stocks.index('MC'))
```

Listes : Méthodes et fonctions

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
print(len(my_stocks))
==> 4
print(my_stocks.append('MSFT'), my_stocks)
==> ['AMZN', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.insert(1, 'FTKE'), my_stocks)
==> None ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META', 'MSFT']
print(my_stocks.count('MSFT'))
==> 1
print(my_stocks.pop(), my_stocks)
==> 'MSFT' ['AMZN', 'FTKE', 'TSLA', 'AAPL', 'META']
print(my_stocks.remove('TSLA'), my_stocks)
==> None ['AMZN', 'FTKE', 'AAPL', 'META']
print(my_stocks.reverse(), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN']
print(my_stocks.extend(['MC', 'TTE']), my_stocks)
==> None ['META', 'AAPL', 'FTKE', 'AMZN', 'MC', 'TTE']
print(my_stocks.sort(reverse=True), my_stocks)
==> None ['TTE', 'META', 'MC', 'FTKE', 'AMZN', 'AAPL']
Print(my_stocks.index('MC'))
==> 2
```

Listes : Approfondissement sur le tri

Il est possible d'effectuer des tris sur des conditions autres que par ordre alphabétique (resp anti-alphabétique) ou croissant (resp décroissant). Ceci s'effectue grâce à l'argument '**key**'. Il est même possible de donner plusieurs critères de tris successifs.

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)  
==> ?
```

```
my_Stocks.sort(key = lambda x : (len(x),x), reverse = False)  
print(my_Stocks)  
==> ?
```

Listes : Approfondissement sur le tri

Il est possible d'effectuer des tris sur des conditions autres que par ordre alphabétique (resp anti-alphabétique) ou croissant (resp décroissant). Ceci s'effectue grâce à l'argument '**key**'. Il est même possible de donner plusieurs critères de tris successifs.

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)  
==> ?
```

```
my_Stocks.sort(key = lambda x : (len(x),x), reverse = False)  
print(my_Stocks)  
==> ?
```

Commentaires

Pour cela il suffit de renseigner une fonction pour l'argument **key** de la méthode **sort** de la classe liste. Il est conseillé d'utiliser une fonction lambda si cette fonction n'a pas vocation à être utilisée ailleurs.

NB : Cette méthode est naturellement utilisable sur des listes de nombres.

Listes : Approfondissement sur le tri

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)
```

Listes : Approfondissement sur le tri

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)  
==> ['TTE', 'AMZN', 'META', 'AAPL', 'FTKE']
```

Tri simple

Ici la fonction lambda désigne la longueur de la chaîne de caractère comme critère de tri.

Listes : Approfondissement sur le tri

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)  
==> ['TTE', 'AMZN', 'META', 'AAPL', 'FTKE']
```

Tri simple

Ici la fonction lambda désigne la longueur de la chaîne de caractère comme critère de tri.

```
my_Stocks.sort(key = lambda x : (len(x),x), reverse = False)  
print(my_Stocks)
```

Listes : Approfondissement sur le tri

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)  
==> ['TTE', 'AMZN', 'META', 'AAPL', 'FTKE']
```

Tri simple

Ici la fonction lambda désigne la longueur de la chaîne de caractère comme critère de tri.

```
my_Stocks.sort(key = lambda x : (len(x),x), reverse = False)  
print(my_Stocks)  
==> ['TTE', 'AAPL', 'AMZN', 'FTKE', 'META']
```

Listes : Approfondissement sur le tri

```
my_Stocks = ['AMZN', 'TTE', 'META', 'AAPL', 'FTKE']  
my_Stocks.sort(key = lambda x : len(x), reverse = False)  
print(my_Stocks)  
==> ['TTE', 'AMZN', 'META', 'AAPL', 'FTKE']
```

Tri simple

Ici la fonction lambda désigne la longueur de la chaîne de caractère comme critère de tri.

```
my_Stocks.sort(key = lambda x : (len(x),x), reverse = False)  
print(my_Stocks)  
==> ['TTE', 'AAPL', 'AMZN', 'FTKE', 'META']
```

Multi-tri

Ici un second critère de tri est utilisé. Attention, Il est dominé par le premier critère. Le code tri d'abord par taille, puis par ordre alphabétique une fois le tri par taille réalisé.

Tuples

Un tuple est un objet de type **container**, les containers contiennent d'autres objets de tous types et ils sont ordonnés.

Déclaration d'un tuple et affichage de ce dernier :

```
my_tuple = tuple()  
my_tuple = ()  
print(my_tuple)
```

Tuples

Un tuple est un objet de type **container**, les containers contiennent d'autres objets de tous types et ils sont ordonnés.

Déclaration d'un tuple et affichage de ce dernier :

```
my_tuple = tuple()  
my_tuple = ()  
print(my_tuple)
```

Caractéristiques

Un tuple est :

- **Ordonné/indexé**
- **Non-Mutable**
- **Accessible**
- **Imbriquable**
- **Extensible**
- **Hashable** (car non-mutable)

Manipuler des tuples

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')  
extra_tickers = ('KO', 'P911')  
print(my_tuple[1], my_tuple[1:3], my_tuple[:-1])
```

Manipuler des tuples

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
extra_tickers = ('KO', 'P911')
print(my_tuple[1], my_tuple[1:3], my_tuple[:-1])
==> AAPL ('AAPL', 'AMZN') ('TTE', 'AAPL', 'AMZN', 'FTKE')

print(hash(extra_tickers))
```

Manipuler des tuples

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
extra_tickers = ('KO', 'P911')
print(my_tuple[1], my_tuple[1:3], my_tuple[:-1])
==> AAPL ('AAPL', 'AMZN') ('TTE', 'AAPL', 'AMZN', 'FTKE')

print(hash(extra_tickers))
==> -6671021492883455079 # valeur variable

extra_tickers = extra_tickers + my_tuple
print(extra_tickers)
```

Manipuler des tuples

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
extra_tickers = ('KO', 'P911')
print(my_tuple[1], my_tuple[1:3], my_tuple[:-1])
==> AAPL ('AAPL', 'AMZN') ('TTE', 'AAPL', 'AMZN', 'FTKE')

print(hash(extra_tickers))
==> -6671021492883455079 # valeur variable

extra_tickers = extra_tickers + my_tuple
print(extra_tickers)
==> ('KO', 'P911', 'TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
print(hash(extra_tickers))
```

Manipuler des tuples

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
extra_tickers = ('KO', 'P911')
print(my_tuple[1], my_tuple[1:3], my_tuple[:-1])
==> AAPL ('AAPL', 'AMZN') ('TTE', 'AAPL', 'AMZN', 'FTKE')

print(hash(extra_tickers))
==> -6671021492883455079 # valeur variable

extra_tickers = extra_tickers + my_tuple
print(extra_tickers)
==> ('KO', 'P911', 'TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
print(hash(extra_tickers))
==> --757245765893052700 # changement du hash de la variable
```

Manipuler des tuples

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
extra_tickers = ('KO', 'P911')
print(my_tuple[1], my_tuple[1:3], my_tuple[:-1])
==> AAPL ('AAPL', 'AMZN') ('TTE', 'AAPL', 'AMZN', 'FTKE')

print(hash(extra_tickers))
==> -6671021492883455079 # valeur variable

extra_tickers = extra_tickers + my_tuple
print(extra_tickers)
==> ('KO', 'P911', 'TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
print(hash(extra_tickers))
==> --757245765893052700 # changement du hash de la variable
```

Un tuple étant **non-mutable** (donc **hashable**), `extra_tickers` n'est plus le même objet après qu'il ait été "modifié", en réalité il a été **ré-instancié**. On le remarque facilement en regardant le **hash** de l'objet. Si le hash d'un objet change entre 2 opérations alors l'objet n'est plus le même, c'est une nouvelle instance.

Tuples : méthodes et fonctions

Les tuples sont une classe d'objets bien plus simples que les listes. Les tuples sont moins flexibles et comportent beaucoup moins de méthodes.

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')  
print(my_tuple.count('TTE'),my_tuple)
```

Tuples : méthodes et fonctions

Les tuples sont une classe d'objets bien plus simples que les listes. Les tuples sont moins flexibles et comportent beaucoup moins de méthodes.

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
print(my_tuple.count('TTE'),my_tuple)
==> 1 ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')

print(my_tuple.index('AMZN'),my_tuple)
```

Tuples : méthodes et fonctions

Les tuples sont une classe d'objets bien plus simples que les listes. Les tuples sont moins flexibles et comportent beaucoup moins de méthodes.

```
my_tuple = ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
print(my_tuple.count('TTE'),my_tuple)
==> 1 ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')

print(my_tuple.index('AMZN'),my_tuple)
==> 2 ('TTE', 'AAPL', 'AMZN', 'FTKE', 'META')
```

Les fonctions usuelles telles que

- len()
- max()
- min()
- ...

fonctionnent sur les tuples.

Exemple

Exemples :
Listes et tuples